

گزارش آزمایشگاه مدارهای منطقی

آزمایش پنجم: واحد محاسبات و منطق (ALU)

علی اصغر طایری — 404171133

رضا حضرتی — 404105767

فهرست مطالب

۳	اول بخش اول: آشنایی با تراشه‌ی 74181 و طراحی مدار ثبات‌ها و کنترل‌کننده
۳	۱ مقدمه و تعریف مسئله
۴	۲ بررسی و انتخاب روش طراحی
۵	۳ بلوک‌دیاگرام طرح پیشنهادی اولیه
۶	۴ روند پیاده‌سازی
۷	۵ دیاگرام یا شماتیک طرح اولیه
۷	۶ دیاگرام طرح نهایی
۷	۷ بررسی <i>Test Case</i> ها
۱۲	۸ نتیجه‌گیری
۱۴	دوم بخش دوم: طراحی <i>ALU</i> چهاربیتی با تراشه‌های <i>TTL</i> پایه
۱۴	۹ مقدمه و تعریف مسئله
۱۴	۱۰ بررسی و انتخاب روش طراحی
۱۵	۱۱ طرح پیشنهادی اولیه
۱۷	۱۲ روند پیاده‌سازی
۱۷	۱.۱۲ بخش اول: واحد جمع و تفریق‌کننده
۱۸	۲.۱۲ بخش دوم: واحد منطقی
۱۸	۳.۱۲ بخش سوم: واحد شیفت
۱۹	۱۳ شماتیک و تصویر مدار نهایی
۲۰	۱۴ بررسی <i>Test Case</i> ها

۲۱		۱.۱۴ عملیات حسابی
۲۲		۲.۱۴ عملیات منطقی
۲۳		۳.۱۴ عملیات شیفت

۲۴		۱۵ نتیجه‌گیری
----	--	---------------

بخش اول: آشنایی با تراشه‌ی 74181 و طراحی مدار ثبات‌ها و کنترل‌کننده

۱ مقدمه و تعریف مسئله

هدف از این آزمایش، آشنایی عملی با واحد محاسبات و منطق (*Arithmetic Logic Unit - ALU*) و بررسی نحوه‌ی عملکرد تراشه‌ی معروف 74181 است. این تراشه یکی از قدیمی‌ترین و شناخته‌شده‌ترین *ALU*های تجاری است که قابلیت انجام عملیات محاسباتی (مانند جمع) و عملیات منطقی (مانند *AND*، *NOT* و غیره) را بر روی دو عملوند ۴ بیتی دارد و انتخاب نوع عملیات از طریق ورودی‌های انتخاب *S3-S0* و ورودی حالت *M* صورت می‌گیرد.

در این آزمایش قرار است مداری طراحی و در نرم‌افزار *Proteus* پیاده‌سازی شود که دارای دو ثبات داده‌ی ۴ بیتی *A* و *B* و یک واحد *ALU* است. مقدار این دو ثبات از طریق دو مالتی‌پلکسر قابل انتخاب است تا هم بتوان مقدار جدیدی از خطوط ورودی *D0-D3* در آن‌ها بارگذاری کرد و هم بتوان خروجی *ALU* را مجدداً به یکی از ثبات‌ها بازگرداند (*feedback*). یک مدار کنترل‌کننده (*Controller*) نیز بر اساس سه ورودی دستور *M0-M2* مشخص می‌کند که در هر لحظه چه عملیاتی روی محتوای ثبات‌ها انجام شود.

جدول عملیات موردنظر که مبنای طراحی کنترل‌کننده قرار گرفته، در جدول ۱ آمده است.

جدول ۱: عملیات صورت‌گرفته در مدار برحسب ورودی‌های *M2-M0*

عملیات	<i>M0</i>	<i>M1</i>	<i>M2</i>
$A \leftarrow D_3 D_2 D_1 D_0$	۰	۰	۰
$B \leftarrow D_3 D_2 D_1 D_0$	۱	۰	۰
$A \leftarrow A$	۰	۱	۰
$A \leftarrow B$	۱	۱	۰
$clear(A)$	۰	۰	۱
$A \leftarrow not(A)$	۱	۰	۱
$A \leftarrow and(A, B)$	۰	۱	۱
$A \leftarrow add(A, B)$	۱	۱	۱

مسئله‌ی اصلی این آزمایش، طراحی بلوک کنترل‌کننده‌ای است که با دریافت سه بیت $M2-M0$ ، سیگنال‌های انتخاب مالتی‌پلکسرهای (SA و SB) و سیگنال‌های انتخاب تابع ALU (یعنی $S3-S0$ و ورودی M) را به‌گونه‌ای تولید کند که هشت عملیات جدول بالا به‌درستی روی ثبات‌های A و B اعمال شوند.

۲ بررسی و انتخاب روش طراحی

برای پیاده‌سازی این مدار، از سه تراشه‌ی اصلی زیر استفاده شد:

- $IC\ 74181$: واحد محاسبات و منطق ۴ بیتی که عملیات محاسباتی و منطقی مورد نیاز جدول ۱ را انجام می‌دهد.
- $IC\ 74175$: چهار فلیپ‌فلاپ D در یک بسته، که برای ساخت دو ثبات ۴ بیتی A و B از دو عدد از این تراشه استفاده شد.
- $IC\ 74157$: مالتی‌پلکسر چهارگانه‌ی ۲ به ۱، که برای انتخاب ورودی هر ثبات (بین داده‌ی خارجی و مقدار بازخوردی) به کار رفت.

با بررسی جدول تابع 74181 (که در دیتاشیت این تراشه موجود است)، مشخص شد برای تولید هر یک از توابع مورد نیاز، ترکیب مشخصی از ورودی حالت M و ورودی‌های انتخاب $S3S2S1S0$ لازم است. جدول ۲ این نگاشت را نشان می‌دهد.

جدول ۲: کدهای انتخاب ALU برای توابع مورد نیاز (طبق دیتاشیت 74181)

تابع مورد نیاز	M	$S3S2S1S0$	Cn	توضیح
$F = A$ (نگهداری A)	۱	۱۱۱۱	-	عبور منطقی A
$F = B$ (انتقال B به A)	۱	۱۰۱۰	-	عبور منطقی B
$F = 0$ ($clear$)	۱	۰۰۱۱	-	صفر منطقی
$F = \bar{A}$ (not)	۱	۰۰۰۰	-	$NOT\ A$
$F = A \cdot B$ (and)	۱	۱۰۱۱	-	AND منطقی
$F = A + B$ (add)	۰	۱۰۰۱	۱	جمع محاسباتی

با توجه به این جدول، روش طراحی انتخاب‌شده به این صورت است: کنترل‌کننده یک مدار ترکیبی (کدکننده) است که ورودی سه‌بیتی $M2-M0$ را به خروجی‌های SA ، SB و $S3-S0$ (به‌همراه ورودی M و Cn تراشه‌ی ALU) ترجمه می‌کند. از آنجا که تعداد حالت‌های ورودی محدود (۸ حالت) و خروجی‌ها نیز مشخص هستند، ساده‌ترین و قابل‌اطمینان‌ترین روش، استخراج مستقیم جدول درستی کنترل‌کننده از روی جدول ۱ و ۲ و سپس پیاده‌سازی آن با

گیت‌های منطقی پایه (NOT ، OR ، AND) بود؛ این روش نسبت به استفاده از ROM یا حافظه، برای مداری با این حجم محدود، از نظر تعداد قطعات و سادگی سیم‌کشی در *Proteus* به‌صرفه‌تر تشخیص داده شد.

۳ بلوک‌دیاگرام طرح پیشنهادی اولیه

بلوک‌دیاگرام کلی مدار پیشنهادی، مطابق شکل داده‌شده در راهنمای آزمایش، شامل دو مالتی‌پلکسر ۴ بیتی در ورودی دو ثبات A و B ، خود دو ثبات A و B ، واحد ALU و بلوک کنترل‌کننده است.

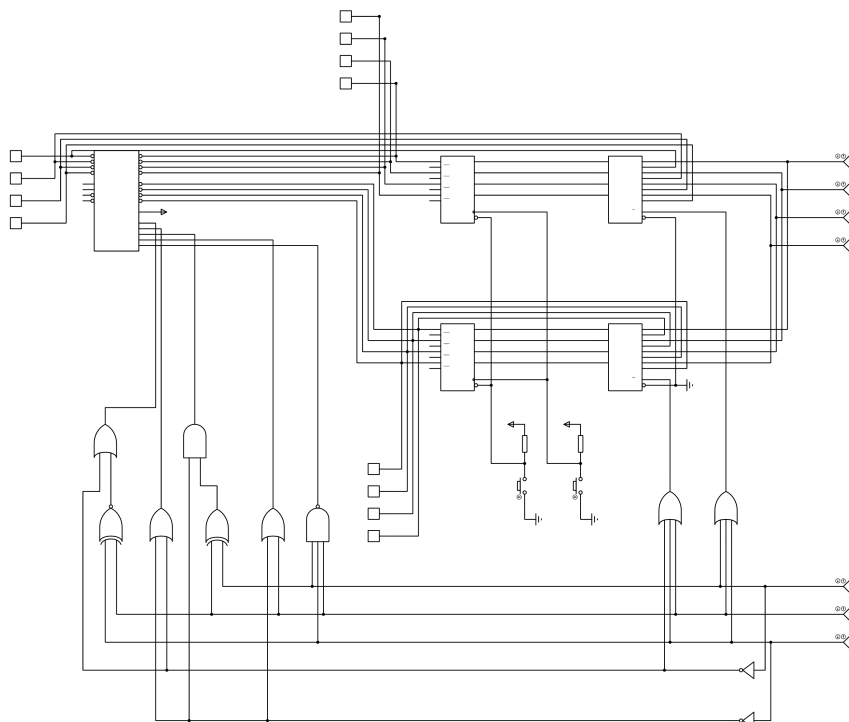
- مالتی‌پلکسر A : بین خطوط ورودی خارجی $D3-D0$ و خروجی F تراشه‌ی ALU (بازخورد) یکی را، بسته به SA ، انتخاب کرده و به ورودی ثبات A می‌دهد.
- مالتی‌پلکسر B : بین خطوط ورودی خارجی $D3-D0$ و خروجی خود ثبات B (برای حفظ مقدار فعلی در حالت‌هایی که B نباید تغییر کند)، بسته به SB ، یکی را انتخاب کرده و به ورودی ثبات B می‌دهد.
- ثبات‌های A و B : با یک پالس ساعت مشترک ($clock$) در هر لبه‌ی ساعت، مقدار ورودی مالتی‌پلکسر مربوط به خود را ذخیره می‌کنند.
- واحد ALU : مقادیر ذخیره‌شده در A و B را دریافت کرده و بسته به $S3-S0$ و M ، خروجی F را تولید می‌کند.
- کنترل‌کننده: با دریافت $M2-M0$ ، سیگنال‌های SA ، SB و $S3-S0$ را طبق جدول ۳ تولید می‌کند.

جدول ۳: جدول درستی کنترل‌کننده

$S0$	$S1$	$S2$	$S3$	M	SB	SA	$M0$	$M1$	$M2$
X	X	X	X	X	۱	۰	۰	۰	۰
X	X	X	X	X	۰	X	۱	۰	۰
۱	۱	۱	۱	۱	۱	۱	۰	۱	۰
۰	۱	۰	۱	۱	۱	۱	۱	۱	۰
۱	۱	۰	۰	۱	۱	۱	۰	۰	۱
۰	۰	۰	۰	۱	۱	۱	۱	۰	۱
۱	۱	۰	۱	۱	۱	۱	۰	۱	۱
۱	۰	۰	۱	۰	۱	۱	۱	۱	۱

در سطرهای اول و دوم که ثبات مقصد فقط بارگذاری از ورودی خارجی است، سایر خروجی‌ها اهمیتی ندارند و در جدول با X (بی‌اهمیت) نشان داده شده‌اند.

شکل ۱ بلوک‌دیاگرام کلی و شماتیک نهایی پیاده‌سازی شده در *Proteus* را نشان می‌دهد.



شکل ۱: شماتیک کلی مدار پیاده‌سازی شده در *Proteus*

۴ روند پیاده‌سازی

پیاده‌سازی مدار طی مراحل زیر انجام شد:

۱. ابتدا دو ثبات A و B با استفاده از دو تراشه‌ی 74175 ساخته شدند. ورودی ساعت (*clock*) هر دو ثبات به یک کلید فشاری (*push-button*) مشترک متصل شد تا با هر بار فشردن، یک لبه‌ی ساعت به هر دو ثبات اعمال شود. همچنین ورودی *Reset* هر دو ثبات به یک کلید فشاری دیگر برای بازگرداندن مدار به حالت اولیه وصل گردید.

۲. در ادامه دو مالتی‌پلکسر با استفاده از تراشه‌ی 74157 در مسیر ورودی هر ثبات قرار داده شدند تا امکان انتخاب بین داده‌ی خارجی و مقدار بازخوردی فراهم شود.

۳. سپس تراشه‌ی 74181 به عنوان *ALU* به خروجی ثبات‌های A و B متصل شد و خروجی F آن هم به عنوان خروجی نهایی مدار (F) و هم به صورت بازخورد به مالتی‌پلکسر ورودی ثبات A وصل گردید.

۴. در نهایت، مدار کنترل‌کننده بر اساس جدول درستی ۳ با گیت‌های منطقی پایه پیاده‌سازی و به ورودی‌های SA ، SB ، $S3-S0$ و M متصل شد. سه کلید (یا سویچ) نیز برای اعمال دستی مقادیر $M2-M0$ در نظر گرفته شد.

۵. پس از اتمام سیم‌کشی، برای هر یک از هشت حالت جدول ۱، مقادیر $M2-M0$ به صورت دستی تنظیم و پس از اعمال چند مقدار نمونه در ورودی‌های $D0-D3$ و فشردن کلید ساعت، صحت مقدار ذخیره‌شده در ثبات‌ها و خروجی ALU با استفاده از پروب‌های ولتاژ (*voltage probe*) در *Proteus* بررسی شد.

در حین پیاده‌سازی، مهم‌ترین نکته‌ی مورد توجه، هم‌زمانی صحیح سیگنال‌های انتخاب (SA ، SB و $S3-S0$) با لبه‌ی ساعت بود؛ زیرا این سیگنال‌ها باید پیش از رسیدن لبه‌ی ساعت به ثبات‌ها، در مالتی‌پلکسرها اعمال شده باشند تا مقدار درستی وارد ثبات شود.

۵ دیاگرام یا شماتیک طرح اولیه

طرح اولیه‌ی پیشنهادی، همان ساختار بلوک دیاگرام ارائه‌شده در راهنمای آزمایش بود: دو مالتی‌پلکسر ۴ بیتی در ورودی ثبات‌های A و B ، خود دو ثبات A و B ، واحد ALU و بلوک کنترل‌کننده، بدون تعیین جزئیات داخلی گیت‌های کنترل‌کننده. در این مرحله صرفاً اتصالات کلی بین بلوک‌ها (مسیر داده‌ی خارجی $D3-D0$ ، مسیر بازخورد از خروجی ALU به مالتی‌پلکسر A ، و اتصال ثبات‌ها به ورودی‌های ALU) مشخص شده بود و طراحی داخلی مدار کنترل‌کننده (یعنی گیت‌هایی که SA ، SB و $S3-S0$ را تولید می‌کنند) در مرحله‌ی بعد، بر اساس جدول ۳، تکمیل شد. این ساختار کلی همان چیزی است که در شکل ۱ (شماتیک نهایی پیاده‌سازی شده) نیز قابل مشاهده است، با این تفاوت که در آن شکل، مدار کنترل‌کننده به طور کامل سیم‌کشی و پیاده‌سازی شده است.

۶ دیاگرام طرح نهایی

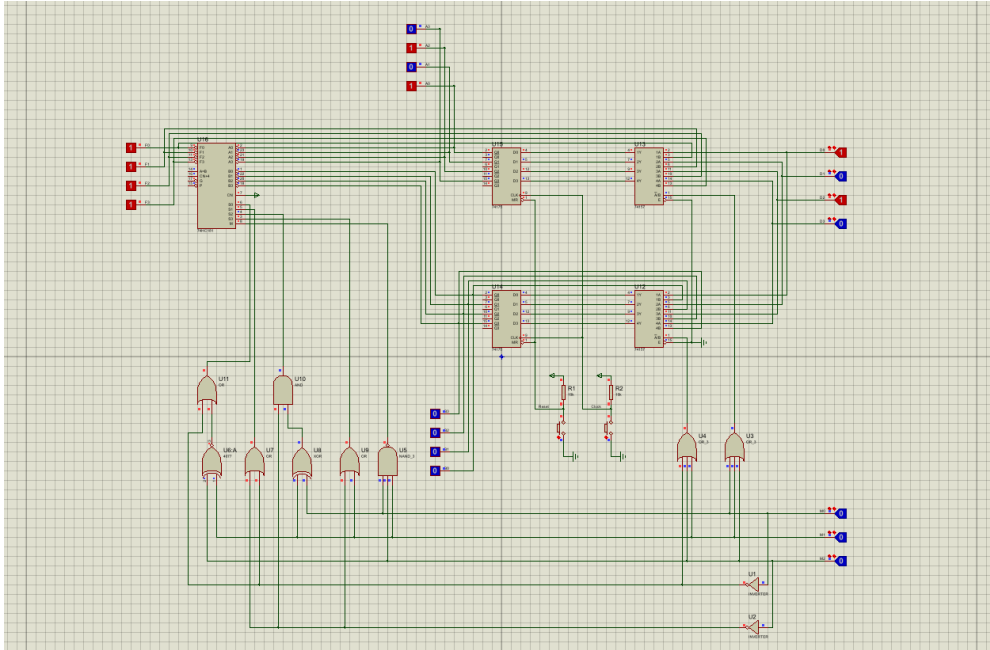
پس از پیاده‌سازی اولیه، در حین تست مدار، ساختار کلی طرح (تعداد و نوع مالتی‌پلکسرها، ثبات‌ها و ALU) بدون تغییر باقی ماند و صرفاً جزئیات داخلی مدار کنترل‌کننده (یعنی گیت‌های تولیدکننده‌ی SA ، SB و $S3-S0$ طبق جدول ۳) تکمیل و سیم‌کشی نهایی شد. به عبارت دیگر، طرح نهایی از نظر بلوک دیاگرام کلی با طرح اولیه یکسان است و تغییری در معماری کلی مدار اعمال نشده است؛ شماتیک نهایی همان شکل ۱ است.

۷ بررسی Test Case ها

پس از تکمیل پیاده‌سازی، مدار برای تمامی هشت حالت ممکن ورودی $M2-M0$ (طبق جدول ۱) آزمایش شد. برای هر حالت، مقادیر مناسبی در ورودی‌های $D0-D3$ اعمال و با فشردن کلید ساعت، نتیجه در خروجی ثبات‌ها و پروب‌های ALU بررسی گردید. در حالت‌های 101 ، 110 و 111 که عملیات آن‌ها به مقدار اولیه‌ی هر دو ثبات A و B وابسته است، دو نمونه‌ی متفاوت آزمایش شد تا صحت عملکرد برای چند ورودی مختلف تأیید شود.

$$\text{حالت } A \leftarrow D_3D_2D_1D_0 : M2M1M0 = 000$$

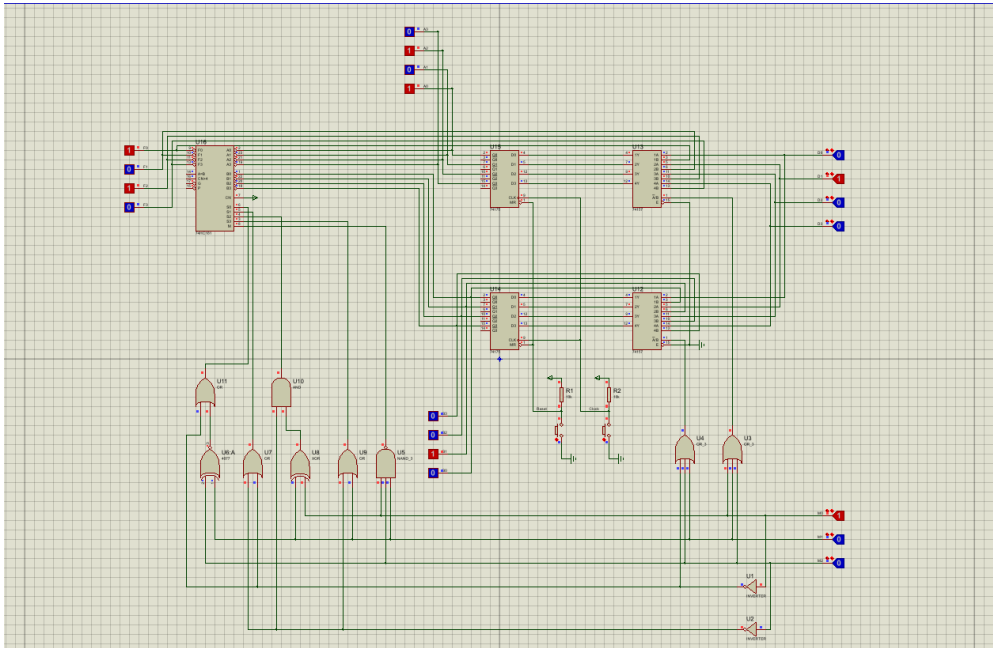
در این حالت مقدار اعمال شده در خطوط $D0-D3$ پس از فشردن کلید ساعت، مستقیماً در ثبات A ذخیره شد؛ همان طور که در شکل ۲ قابل مشاهده است.



شکل ۲: نتیجه‌ی تست حالت 000

$$\text{حالت } B \leftarrow D_3D_2D_1D_0 : M2M1M0 = 001$$

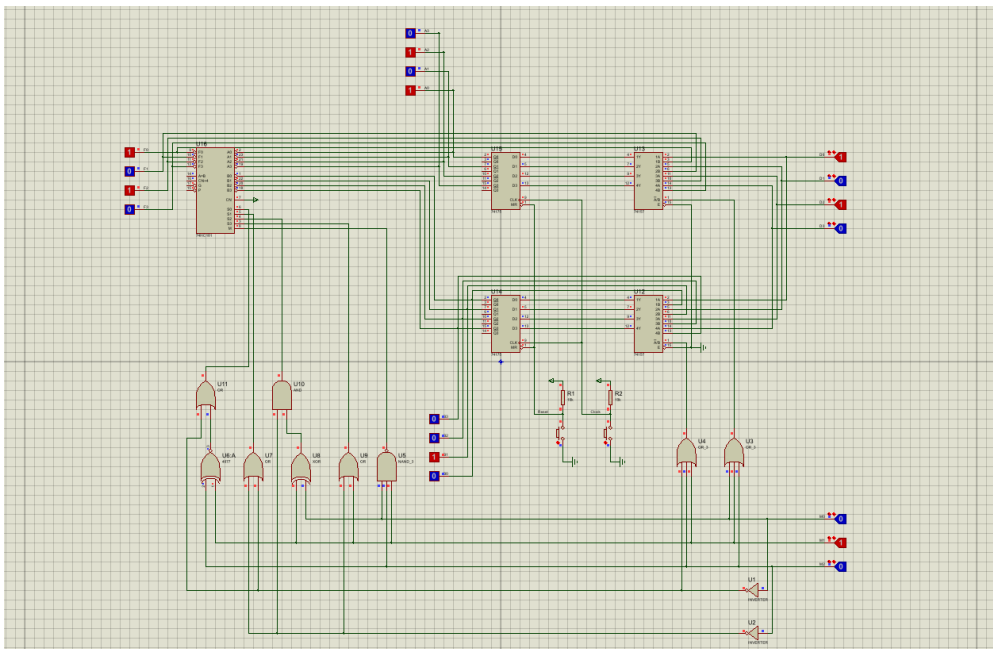
در این حالت مقدار خطوط $D0-D3$ در ثبات B بارگذاری شد و ثبات A بدون تغییر باقی ماند (شکل ۳).



شکل ۳: نتیجه‌ی تست حالت 001

حالت $A \leftarrow A : M2M1M0 = 010$

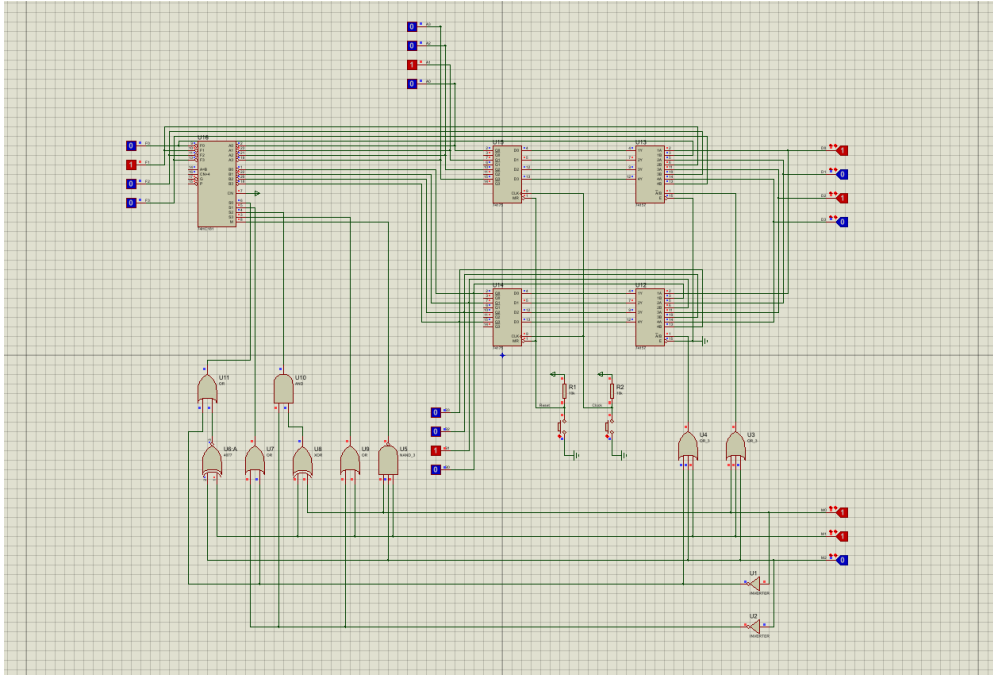
در این حالت مقدار ثبات A بدون تغییر باقی ماند و پس از فشردن کلید ساعت نیز همان مقدار قبلی در A حفظ شد (شکل ۴).



شکل ۴: نتیجه‌ی تست حالت 010

حالت $A \leftarrow B : M2M1M0 = 011$

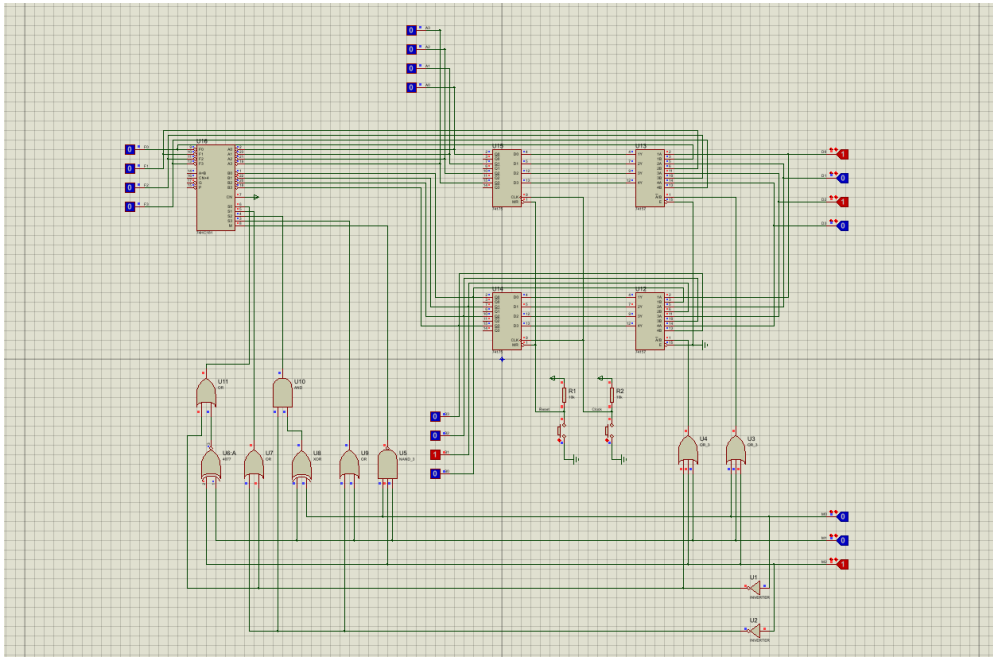
مقدار ذخیره شده در ثبات B در این حالت به ثبات A منتقل شد (شکل ۵).



شکل ۵: نتیجه‌ی تست حالت 011

حالت $clear(A) : M2M1M0 = 100$

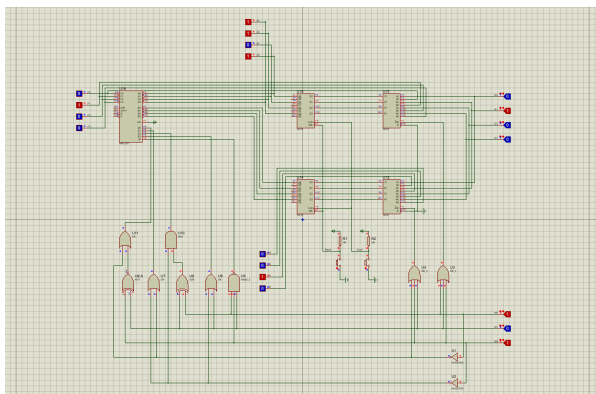
در این حالت با فشردن کلید ساعت، ثبات A صفر شد (شکل ۶).



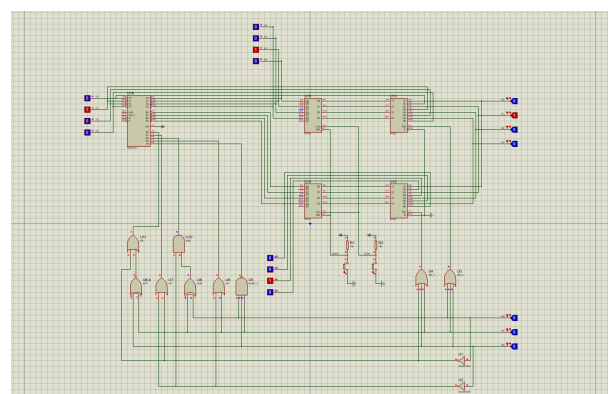
شکل ۶: نتیجه‌ی تست حالت 100

حالت $A \leftarrow not(A) : M2M1M0 = 101$

مقدار متمم ثبات A در آن ذخیره شد. این حالت برای دو مقدار اولیه‌ی متفاوت آزمایش شد (شکل ۷).



(ب) نمونه‌ی دوم

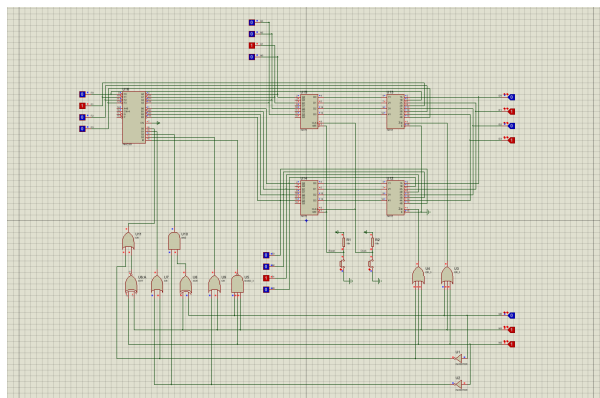


(آ) نمونه‌ی اول

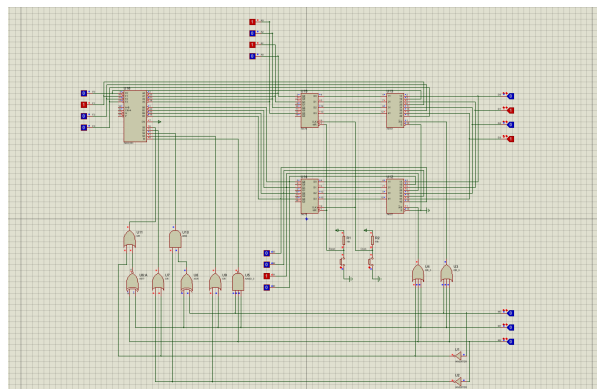
شکل ۷: نتیجه‌ی تست حالت 101

حالت $A \leftarrow and(A, B) : M2M1M0 = 110$

حاصل AND منطقی بین A و B در ثبات A ذخیره شد. این حالت نیز برای دو نمونه‌ی مختلف بررسی شد (شکل ۸).



(ب) نمونه‌ی دوم

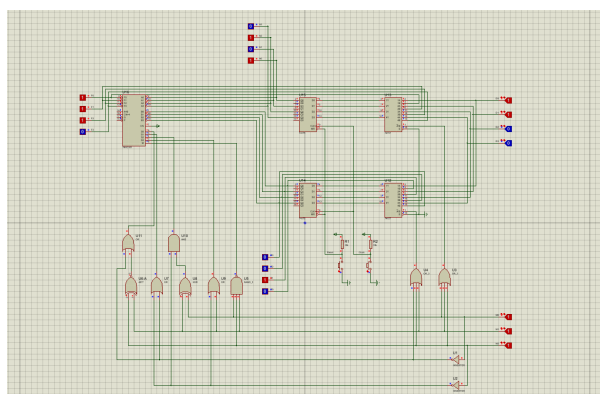


(آ) نمونه‌ی اول

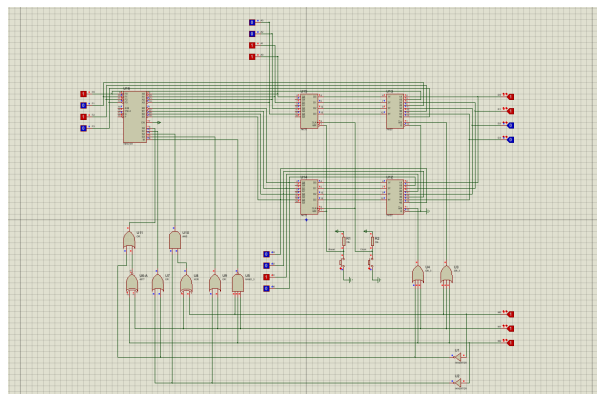
شکل ۸: نتیجه‌ی تست حالت 110

حالت $A \leftarrow add(A, B) : M2M1M0 = 111$

حاصل جمع محاسباتی A و B در ثبات A ذخیره شد. این حالت نیز برای دو نمونه‌ی ورودی مختلف، ازجمله بررسی تولید یا عدم تولید رقم نقلی، آزمایش شد (شکل ۹).



(ب) نمونه‌ی دوم



(آ) نمونه‌ی اول

شکل ۹: نتیجه‌ی تست حالت 111

نتایج به‌دست‌آمده از تمامی هشت حالت فوق، کاملاً منطبق بر جدول عملیات ۱ بود و هیچ‌گونه ناسازگاری‌ای بین رفتار موردانتظار و رفتار مشاهده‌شده در شبیه‌سازی *Proteus* مشاهده نشد.

۸ نتیجه‌گیری

در این آزمایش، با ساخت و شبیه‌سازی مداری شامل تراشه‌ی 74181 (ALU)، دو تراشه‌ی 74175 (ثبات) و دو تراشه‌ی 74157 (مالتی‌پلکسر)، نحوه‌ی کنترل عملیات محاسباتی و منطقی روی دو ثبات به‌صورت عملی بررسی شد. مهم‌ترین نکته‌ی آموخته‌شده، اهمیت طراحی صحیح مدار کنترل‌کننده برای تبدیل یک کد دستور کوتاه (سه‌بیتی) به سیگنال‌های

انتخاب مناسب ALU و مالتی پلکسرهاست؛ این ساختار در واقع نمونه‌ای ساده از معماری بخش کنترل و مسیر داده ($datapath$ و $control\ unit$) در پردازنده‌های واقعی است.

آزمایش تمامی هشت حالت جدول عملیات نشان داد که مدار طراحی شده به درستی و مطابق انتظار عمل می‌کند و خروجی‌های به دست آمده در تمام موارد با مقادیر پیش‌بینی شده مطابقت دارد. این آزمایش درک بهتری از کاربرد عملی تراشه‌ی 74181 به عنوان یک واحد محاسبات و منطق کلاسیک، و نیز نحوه‌ی طراحی مدارهای کنترل‌کننده‌ی ترکیبی برای سیستم‌های دیجیتال کوچک فراهم کرد.

بخش دوم: طراحی ALU چهاربیتی با تراشه‌های TTL

پایه

۹ مقدمه و تعریف مسئله

هدف از آزمایش پنجم، آشنایی عملی با نحوه‌ی طراحی و پیاده‌سازی یک واحد محاسبات و منطق (*Arithmetic Logic Unit* یا به اختصار ALU) است. ALU یکی از اصلی‌ترین اجزای هر واحد پردازش دیجیتال (مانند پردازنده‌های مرکزی) محسوب می‌شود که وظیفه‌ی انجام عملیات حسابی (جمع، تفریق، افزایش، کاهش و مانند آن) و عملیات منطقی (XOR ، OR ، AND و متمم) و همچنین عملیات جابه‌جایی بیت‌ها (شیفت) را بر عهده دارد. تمامی مراحل طراحی، شبیه‌سازی و آزمایش این مدار با استفاده از نرم‌افزار *Proteus* انجام شده است.

مسئله‌ای که در این آزمایش باید حل شود به این صورت تعریف می‌شود: یک ALU چهاربیتی طراحی و پیاده‌سازی شود که دارای ورودی‌های زیر باشد:

- دو عملوند چهاربیتی A ($A0-A3$) و B ($B0-B3$)

- یک ورودی کری Cin

- چهار خط انتخاب عملیات ($S0-S3$) $Select$

و دارای خروجی‌های زیر باشد:

- یک خروجی چهاربیتی F ($F0-F3$)

- یک خروجی کری خروجی $Cout$

با توجه به مقدار خطوط انتخاب $S0 S1 S2 S3$ و ورودی Cin ، مدار باید یکی از چهارده عملیات مشخص شده در جدول عملکرد (جدول ۴) را روی عملوندهای ورودی اجرا کرده و نتیجه را در خروجی F (و در حالت عملیات حسابی، کری خروجی را در $Cout$) قرار دهد. در پایان آزمایش نیز باید صحت عملکرد مدار برای تمامی حالت‌های ممکن خط انتخاب، توسط ابزار *Logic Probe* در محیط *Proteus* بررسی و مستند شود.

۱۰ بررسی و انتخاب روش طراحی

برای ساخت مدار داخلی یک ALU (که یک واحد محاسبات و منطق است) به‌جای طراحی مدار کاملاً سفارشی از دروازه‌های منطقی پایه، از رویکرد استفاده از تراشه‌های TTL آماده بهره گرفته شده است؛ چراکه این روش هم پیاده‌سازی

را ساده‌تر می‌کند و هم با تجهیزات آزمایشگاهی و کتابخانه‌ی *Proteus* سازگاری کامل دارد. اجزای اصلی به‌کاررفته عبارت‌اند از:

- یک تراشه 74283 (جمع‌کننده‌ی کامل چهاربیتی) به‌عنوان هسته‌ی محاسباتی جمع/تفریق؛
- شش تراشه 74153 (مالتی پلکسر دوگانه‌ی چهاربه‌یک) برای انتخاب عملوندها و ترکیب نتایج بخش‌های مختلف؛
- دروازه‌های 7408 (AND)، 7432 (OR)، 74LS86 (XOR) و 7404 (NOT) به تعداد کافی برای پیاده‌سازی واحد منطقی و تولید سیگنال‌های کمکی (مانند $\bar{A}$ و $\bar{B}$).

دلیل انتخاب این روش آن است که تراشه‌ی 74283 به‌طور مستقیم عملیات جمع چهاربیتی همراه با کری ورودی/خروجی را انجام می‌دهد و با تغذیه‌ی مناسب ورودی دوم آن (به‌جای B ساده) می‌توان تمامی هشت عملیات حسابی جدول را — بدون نیاز به مدار جمع/تفریق‌کننده‌ی جداگانه — تنها با انتخاب مقدار مناسب برای این ورودی (توسط مالتی پلکسر) پیاده‌سازی کرد. همچنین استفاده از مالتی پلکسرهای 74153 در سطوح بعدی، امکان می‌دهد که سه بخش مستقل مدار (حسابی، منطقی و شیفت) به‌صورت جداگانه طراحی شوند و در نهایت با یک لایه‌ی انتخاب نهایی، بر اساس خطوط S_2 و S_3 ، خروجی مناسب انتخاب گردد. این تفکیک، طراحی، اشکال‌زدایی و سیم‌کشی مدار را به‌مراتب ساده‌تر از یک پیاده‌سازی یکپارچه می‌کند.

به‌طور کلی مدار به سه بخش اصلی تقسیم می‌شود:

۱. واحد جمع و تفریق‌کننده (عملیات حسابی)

۲. واحد منطق پایه (AND ، OR ، XOR ، متمم)

۳. واحد شیفت به راست و چپ

که خروجی هر یک از این سه بخش، به‌همراه سیگنال‌های انتخاب S_2 و S_3 ، به مالتی پلکسرهای نهایی داده می‌شود تا در نهایت یکی از آن‌ها به‌عنوان خروجی F انتخاب شود. جزئیات کامل هر بخش در قسمت «روند پیاده‌سازی» آمده است.

۱۱ طرح پیشنهادی اولیه

پیش از ورود به جزئیات مدار و سیم‌کشی نهایی، ساختار کلی سیستم به‌صورت مفهومی و بر پایه‌ی تحلیل نظری تعریف شد. طبق این طرح اولیه، ALU چهاربیتی باید دارای دو ورودی عملوند چهاربیتی A و B ، یک ورودی کری Cin و یک ورودی انتخاب چهاربیتی $Select$ باشد و در خروجی، نتیجه‌ی چهاربیتی F به‌همراه کری خروجی $Cout$ تولید کند. از آنجا که در این مرحله صرفاً ساختار کلی و تقسیم‌بندی وظایف مدار مشخص می‌شد — و نه جزئیات سیم‌کشی یا چیدمان قطعات در محیط *Proteus* — هیچ شماتیک یا بلوک‌دیاگرام تصویری از این مرحله تهیه نشده است؛ آنچه در ادامه می‌آید، شرح متنی همین طرح مفهومی اولیه است.

بر اساس این تحلیل اولیه، مقرر شد مدار داخلی ALU به سه زیرواحد مستقل تقسیم شود:

۱. زیرواحد حسابی، مسئول انجام هشت عملیات جمع، تفریق، افزایش و کاهش؛

۲. زیرواحد منطقی، مسئول انجام چهار عملیات AND ، OR ، XOR و متمم؛

۳. زیرواحد شیفت، مسئول جابه‌جایی بیت‌های A به راست یا چپ.

و در نهایت، خروجی هر یک از این سه زیرواحد، توسط یک لایه‌ی مالتی‌پلکسر انتخاب نهایی — بر اساس مقدار خطوط $S2$ و $S3$ — به‌عنوان خروجی F مدار انتخاب گردد. این تقسیم‌بندی مفهومی، مبنای طراحی و سیم‌کشی مدار در بخش «روند پیاده‌سازی» قرار گرفت.

جدول عملکرد کامل که مبنای طراحی این آزمایش قرار گرفته، در جدول ۴ آمده است. همان‌طور که دیده می‌شود، ترکیب خطوط انتخاب $S2$ $S3$ تعیین می‌کند که مدار در کدام یک از سه حالت (حسابی، منطقی یا شیفت) قرار گیرد و در ادامه، $S0$ $S1$ (و در حالت حسابی، Cin) نیز عملیات دقیق درون آن حالت را مشخص می‌کنند.

جدول ۴: جدول عملکرد ALU چهاربیتی (مبنای طراحی)

توضیح	عملیات	C_{in}	S_0	S_1	S_2	S_3
انتقال A	$F = A$	0	0	0	0	0
افزایش (اینکریمنت) A	$F = A + 1$	1	0	0	0	0
جمع	$F = A + B$	0	1	0	0	0
جمع با کarry	$F = A + B + 1$	1	1	0	0	0
تفریق با قرض	$F = A + \overline{B}$	0	0	1	0	0
تفریق	$F = A + \overline{B} + 1$	1	0	1	0	0
کاهش (دیکرمنت) A	$F = A - 1$	0	1	1	0	0
انتقال A	$F = A$	1	1	1	0	0
AND	$F = A \wedge B$	×	0	0	1	0
OR	$F = A \vee B$	×	1	0	1	0
XOR	$F = A \oplus B$	×	0	1	1	0
متمم A	$F = \overline{A}$	×	1	1	1	0
شیفت به راست A در F	$F = \text{shr } A$	×	×	×	0	1
شیفت به چپ A در F	$F = \text{shl } A$	×	×	×	1	1

۱۲ روند پیاده‌سازی

پیاده‌سازی مدار در سه گام مطابق با تفکیک طرح اولیه انجام شد.

۱.۱۲ بخش اول: واحد جمع و تفریق‌کننده

در این بخش رابطه‌ی کلی زیر پیاده‌سازی می‌شود:

$$F = A + X + C_{in}$$

که در آن X می‌تواند B ، متمم B (یعنی $\overline{B}$)، مقدار صفر یا مقدار یک (تمام بیت‌ها) باشد. برای انتخاب این چهار حالت از دو تراشه‌ی مالتی‌پلکسر 74153 استفاده شده است (از آنجا که ورودی‌ها و خروجی‌ها چهاربیتی هستند و هر تراشه‌ی 74153 از دو مالتی‌پلکسر چهاربه‌یک تشکیل شده، به دو تراشه یعنی چهار واحد مالتی‌پلکسر نیاز است تا هر بیت از X به‌طور مستقل انتخاب شود).

پایه‌های ورودی هر مالتی‌پلکسر به‌ترتیب به تغذیه (VCC)، B ، متمم B و زمین (GND) متصل شده‌اند. سپس بیت‌های $A0$ تا $A3$ به ورودی‌های $A0$ تا $A3$ تراشه‌ی 74283 و بیت‌های $B0$ تا $B3$ این تراشه به خروجی‌های چهار مالتی‌پلکسر (یعنی مقدار انتخاب‌شده‌ی X) متصل می‌شوند. پایه‌های انتخاب مالتی‌پلکسرهای نیز به $S0$ و $S1$ وصل شده‌اند تا مطابق جدول عملکرد، یکی از چهار حالت X انتخاب شود.

در خروجی تراشه‌ی جمع‌کننده، کری خروجی ($Cout$) مستقیماً به $Logic Probe$ متصل می‌شود، اما خروجی‌های اصلی جمع ($F0$ تا $F3$) باید به ورودی اول دو مالتی‌پلکسر نهایی (که در بخش بعد معرفی می‌شوند) داده شوند تا در انتخاب نهایی خروجی F شرکت کنند.

۲.۱۲ بخش دوم: واحد منطقی

در این بخش ابتدا چهار عملیات پایه‌ی $A \oplus B$ ، $A \vee B$ ، $A \wedge B$ و $\overline{A}$ با استفاده از گیت‌های 7408، 7432، 74LS86 و 7404 ساخته می‌شوند. سپس با استفاده از دو تراشه‌ی مالتی‌پلکسر دیگر، یکی از این چهار نتیجه انتخاب می‌شود: ورودی اول هر مالتی‌پلکسر به خروجی گیت AND ، ورودی دوم به OR ، ورودی سوم به XOR و ورودی چهارم به گیت NOT (متمم A) متصل است. پایه‌های انتخاب این مالتی‌پلکسرهای نیز مانند بخش قبل به $S0$ و $S1$ وصل شده‌اند. خروجی این دو مالتی‌پلکسر به ورودی دوم مالتی‌پلکسرهای نهایی داده می‌شود.

۳.۱۲ بخش سوم: واحد شیفیت

برای این بخش نیازی به گیت اضافه نیست و شیفیت به‌صورت مستقیم با سیم‌کشی مناسب ورودی‌های A پیاده‌سازی می‌شود؛ برای شیفیت به راست، بیت $A0$ به زمین وصل شده و سایر بیت‌ها به‌ترتیب یک واحد جابه‌جا می‌شوند و به همین ترتیب برعکس، برای شیفیت به چپ. این سیم‌کشی مستقیماً به ورودی سوم و چهارم مالتی‌پلکسرهای نهایی متصل می‌شود.

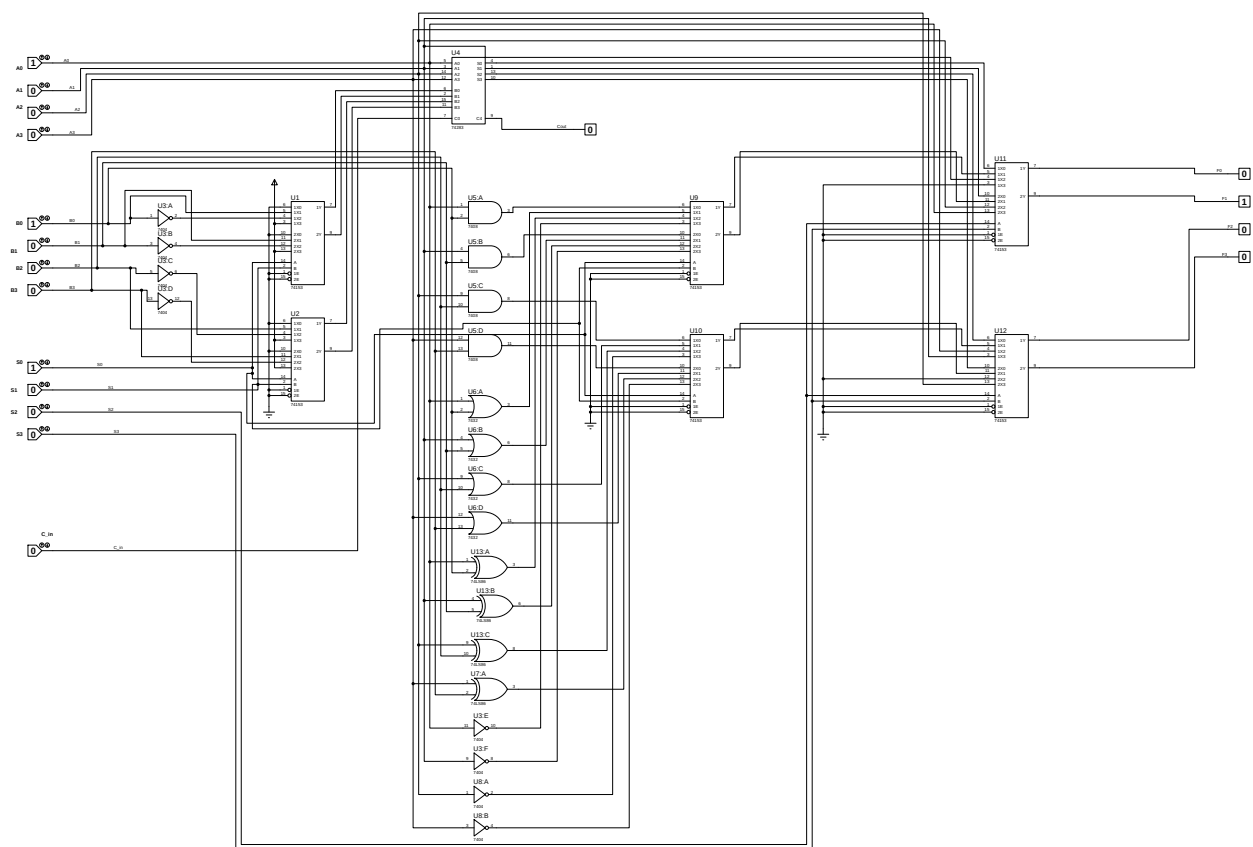
پایه‌های انتخاب دو مالتی‌پلکسر نهایی به $S2$ و $S3$ وصل شده‌اند تا در نهایت یکی از چهار مسیر (حسابی، منطقی، شیفیت راست، شیفیت چپ) به عنوان خروجی F انتخاب و به $Logic Probe$ ‌های خروجی نمایش داده شود.

با توجه به این‌که پس از پیاده‌سازی، طرح اولیه به‌طور کامل پاسخ‌گوی نیاز آزمایش بود، نسخه‌ی اولیه و نهایی این مدار هیچ تفاوتی با یکدیگر نداشتند و نیازی به هیچ تغییری در ساختار یا سیم‌کشی مدار پیش نیامد.

۱۳ شماتیک و تصویر مدار نهایی

لازم به ذکر است که در این آزمایش، طراحی مدار مستقیماً بر پایه‌ی طرح مفهومی اولیه (بخش «طرح پیشنهادی اولیه») و جدول عملکرد جدول ۴ انجام شد و در همان تلاش نخست، بدون نیاز به هیچ بازبینی یا اصلاح، به درستی پاسخ‌گوی تمامی حالت‌ها بود؛ به عبارت دیگر، این مدار اساساً مرحله یا نسخه‌ی «اولیه»ی جداگانه‌ای (متفاوت از طرح نهایی) نداشته است. به همین دلیل هیچ تصویر یا شماتیکی از یک نسخه‌ی «اولیه» در دسترس نیست؛ آنچه در ادامه ارائه می‌شود – هم شماتیک و هم تصویر واقعی مدار – هر دو متعلق به همان یک نسخه‌ی طراحی شده و نهایی مدار، در محیط *Proteus*، هستند.

شماتیک کامل مدار نهایی، شامل تراشه‌ی جمع‌کننده‌ی ۷۴۲۸۳ (با نام $U4$)، شش مالتی‌پلکسر ۷۴۱۵۳ ($U1, U2, U9, U10, U11$ و $U12$)، دروازه‌های ($U5$) AND، ($U6$) OR، ($U13$ و $U7$) XOR و معکوس‌کننده‌های ($U3$) NOT و ($U8$)، در شکل ۱۰ نشان داده شده است.



شکل ۱۰: شماتیک کامل مدار نهایی ALU چهاربیتی در محیط *Proteus*

همان‌طور که در شکل مشخص است، ورودی‌های $A0-A3$ ، $B0-B3$ ، $S0-S3$ و Cin در سمت چپ مدار قرار دارند و خروجی‌های $F0-F3$ و $Cout$ در سمت راست مدار نمایش داده می‌شوند. ساختار سه‌بخشی مدار (واحد حسابی در بالا، واحد منطقی در وسط و واحد شیفت به صورت سیم‌کشی مستقیم به مالتی‌پلکسرهای نهایی) در این شماتیک کاملاً قابل مشاهده است.

۱۴ بررسی Test Case ها

برای بررسی صحت عملکرد مدار، تمامی چهارده حالت جدول عملکرد (جدول ۴) به صورت جداگانه در Proteus تحریک شدند و مقادیر Logic Probe های متصل به $F0-F3$ و Cout برای هر حالت ثبت شد. نامگذاری فایل های تصویر هر Test Case بر اساس ترتیب $S_3S_2S_1S_0C_{in}$ انجام شده است (علامت x به معنای بی اثر بودن آن بیت در عملیات مورد نظر است). جدول ۵ نگاشت میان نام هر فایل تصویر و عملیات متناظر با آن را نشان می دهد.

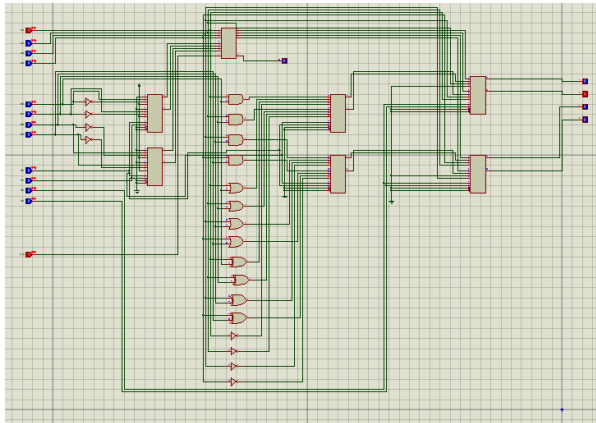
جدول ۵: نگاشت Test Case ها به عملیات جدول عملکرد

نام فایل تصویر	$S_3S_2S_1S_0C_{in}$	عملیات
00000	0 0 0 0 0	$F = A$ (انتقال A)
00001	0 0 0 0 1	$F = A + 1$ (افزایش)
00010	0 0 0 1 0	$F = A + B$ (جمع)
00011	0 0 0 1 1	$F = A + B + 1$ (جمع با کری)
00100	0 0 1 0 0	$F = A + \overline{B}$ (تفریق با قرض)
00101	0 0 1 0 1	$F = A + \overline{B} + 1$ (تفریق)
00110	0 0 1 1 0	$F = A - 1$ (کاهش)
00111	0 0 1 1 1	$F = A$ (انتقال A)
0100x	0 1 0 0 x	$F = A \wedge B$ (AND)
0101x	0 1 0 1 x	$F = A \vee B$ (OR)
0110x	0 1 1 0 x	$F = A \oplus B$ (XOR)
0111x	0 1 1 1 x	$F = \overline{A}$ (متمم A)
10xxx	1 0 x x x	$F = \text{shr } A$ (شیفت راست)
11xxx	1 1 x x x	$F = \text{shl } A$ (شیفت چپ)

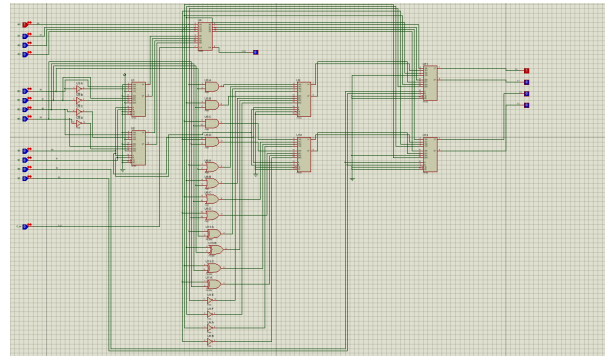
پیش از بررسی تک تک حالت ها، نمای کلی مدار نهایی پیاده سازی شده به همراه محل قرارگیری کلیدهای ورودی و Logic Probe های خروجی پیش تر در شکل ۱۰ (بخش «شماتیک و تصویر مدار نهایی») نشان داده شد.

۱.۱۴ عملیات حسابی

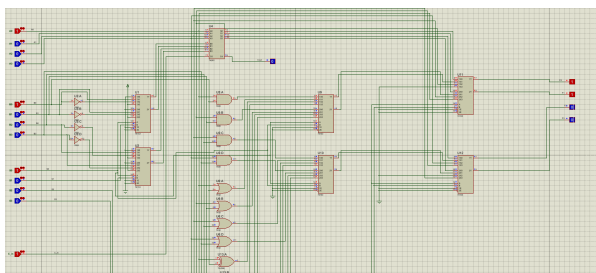
هشت حالت اول جدول عملکرد مربوط به عملیات حسابی هستند که با $S_3S_2 = 00$ و ترکیب‌های مختلف $S_1S_0C_{in}$ فعال می‌شوند. نتایج شبیه‌سازی برای این هشت حالت در شکل‌های ۱۱ و ۱۲ نشان داده شده‌اند.



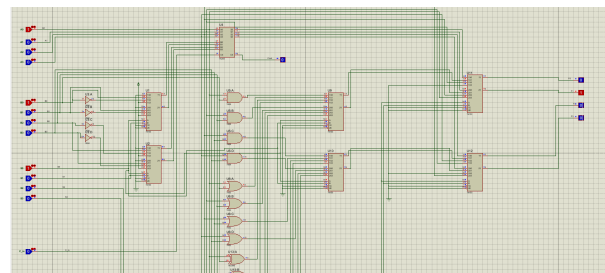
$F = A + 1$ (ب)



$F = A$ (آ)

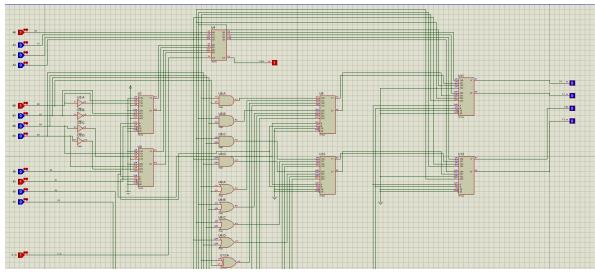


$F = A + B + 1$ (د)

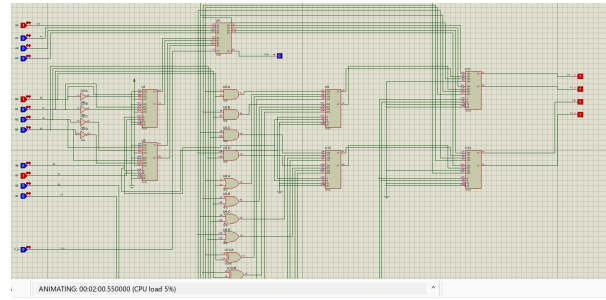


$F = A + B$ (ج)

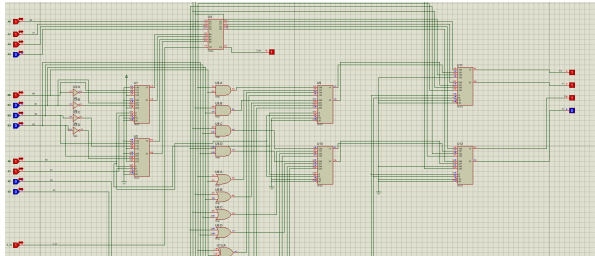
شکل ۱۱: نتیجه‌ی شبیه‌سازی چهار عملیات حسابی نخست



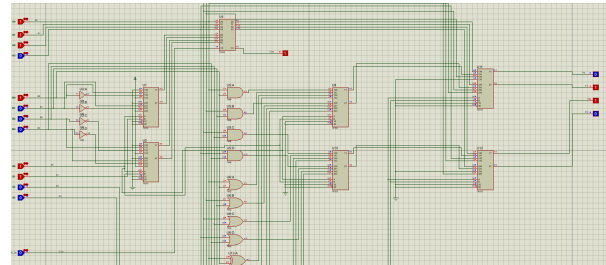
$$F = A + \overline{B} + 1 \text{ (ب)}$$



$$F = A + \overline{B} \text{ (ا)}$$



$$F = A \text{ (د)}$$



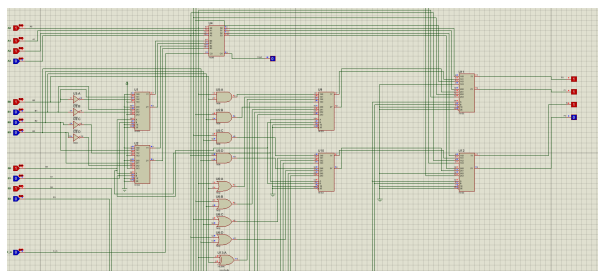
$$F = A - 1 \text{ (ج)}$$

شکل ۱۲: نتیجه‌ی شبیه‌سازی چهار عملیات حسابی دوم

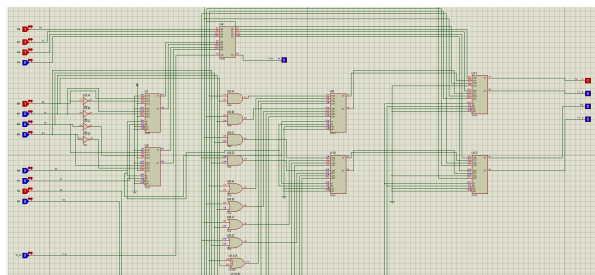
در تمامی هشت حالت فوق، مقدار خوانده‌شده از *Logic Probe* های $F0-F3$ با مقدار محاسبه‌شده‌ی دستی از روی جدول عملکرد یکسان بود و کری خروجی (*Cout*) نیز به‌درستی وضعیت سرریز جمع/تفریق را نشان می‌داد.

۲.۱۴ عملیات منطقی

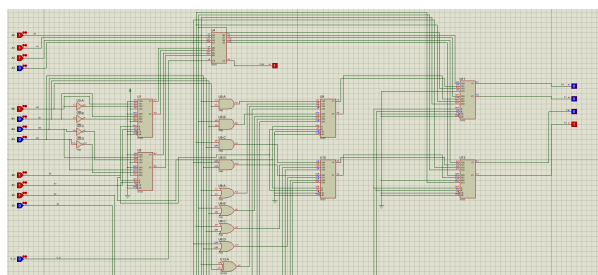
چهار عملیات منطقی با $S_3S_2 = 01$ و مقادیر مختلف S_1S_0 فعال می‌شوند (مقدار Cin در این حالت بی‌اثر است). نتایج در شکل ۱۳ نشان داده شده‌اند.



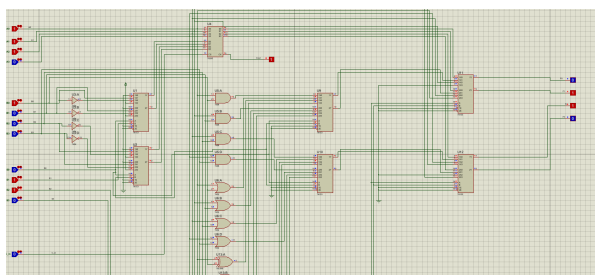
(ب) $F = A \vee B$ (OR)



(AND) $F = A \wedge B$ ($\bar{I}$)



(د) $F = \bar{A}$ (متمم)



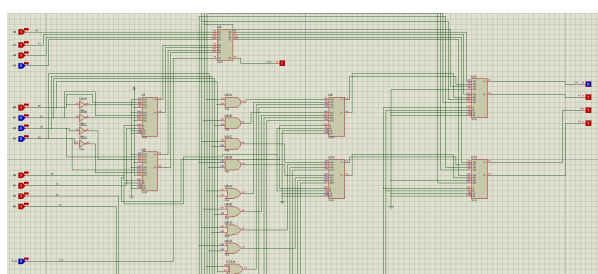
(ج) $F = A \oplus B$ (XOR)

شکل ۱۳: نتیجه‌ی شبیه‌سازی چهار عملیات منطقی

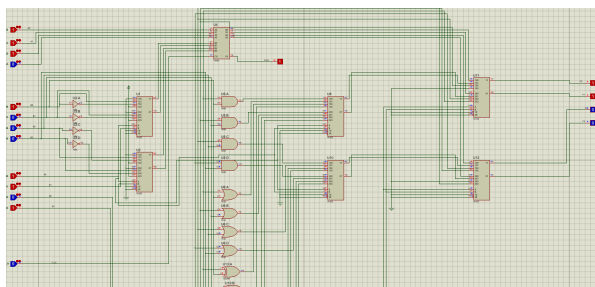
در هر چهار حالت، خروجی F دقیقاً برابر با نتیجه‌ی منطقی مورد انتظار روی عملوندهای A و B بود و همان‌طور که از جدول عملکرد انتظار می‌رفت، مقدار $Cout$ در این حالت فاقد معنای مشخص (بی‌اثر) است.

۳.۱۴ عملیات شیفت

دو عملیات پایانی جدول، شیفت به راست و شیفت به چپ A هستند که با $S_3S_2 = 10$ و $S_3S_2 = 11$ به ترتیب فعال می‌شوند. نتایج در شکل ۱۴ آمده است.



(ب) $F = \text{shl } A$



($\bar{I}$) $F = \text{shr } A$

شکل ۱۴: نتیجه‌ی شبیه‌سازی دو عملیات شیفت

در هر دو حالت، بیت جدیدی که از سمت آزاد وارد F می‌شود برابر با صفر بود و سایر بیت‌های A دقیقاً یک واحد به سمت مربوطه جابه‌جا شدند که کاملاً منطبق با تعریف عملیات شیفت منطقی در جدول عملکرد است.

۱۵ نتیجه‌گیری

در این آزمایش یک واحد محاسبات و منطق (*ALU*) چهاربیتی، مطابق با جدول عملکرد استاندارد (چهارده عملیات حسابی، منطقی و شیفت)، با استفاده از تراشه‌ی جمع‌کننده‌ی 74283، شش تراشه‌ی مالتی‌پلکسر 74153 و تعداد کافی گیت‌های *AND*، *OR*، *XOR* و *NOT* طراحی و در نرم‌افزار *Proteus* پیاده‌سازی شد.

تفکیک مدار به سه زیرواحد مستقل حسابی، منطقی و شیفت – که در نهایت توسط یک لایه‌ی مالتی‌پلکسر انتخاب می‌شدند – روش طراحی را به‌طور قابل‌توجهی ساده‌تر و قابل‌فهم‌تر کرد و نیاز به هیچ مدار جمع/تفریق‌کننده یا واحد منطقی جداگانه‌ای نبود؛ تمام عملیات حسابی صرفاً با تغییر ورودی دوم تراشه‌ی جمع‌کننده (توسط مالتی‌پلکسرهای سطح اول) به‌دست آمدند.

با اعمال تمامی چهارده *Test Case* ممکن (مطابق جدول ۵)، خروجی مدار در همه‌ی حالت‌ها دقیقاً با مقدار موردانتظار از روی جدول عملکرد مطابقت داشت و نیازی به اصلاح طرح اولیه پیش نیامد؛ به همین دلیل طرح اولیه هیچ تفاوتی با طرح نهایی نداشت و شماتیک و تصویر ارائه‌شده در این گزارش، هر دو متعلق به همان مدار نهایی پیاده‌سازی شده هستند. مهم‌ترین نکته‌ی آموخته‌شده در این آزمایش، توانایی ترکیب چند تراشه‌ی *MSI* (نظیر جمع‌کننده و مالتی‌پلکسر) به‌جای طراحی مدار از پایه با دروازه‌های منطقی است؛ رویکردی که در طراحی واقعی پردازنده‌ها نیز به‌طور گسترده استفاده می‌شود.